Understanding ML, Chapter 3: From Fitting to Generalization

Why training performance lies, what capacity, bias and variance actually are, what regularisation, feature selection, early stopping and the split protocols manipulate, and the experiments that reveal each one

Fundamentals of Machine Learning, midterm

2026-11-03

What this chapter is about, in one paragraph

Chapter 2 taught fitting: pick the parameters that make the training examples look good. This chapter is about the gap between looking good on the training examples and being good on the next case. It names the gap (generalisation gap), proves that training performance is optimistic, shows that the cause is capacity (a flexible family can fit the sample's accidents), splits the error at one input into noise, bias and variance, and then goes through every dial that controls effective capacity: penalties (L_2 shrinks, L_1 zeroes), early stopping, feature selection. It ends with the rules that make an evaluation honest: a split that matches deployment, metrics that expose minority failure, no leakage. Every method here is an answer to one question: *which of the regularities in my sample will still hold on the next case?* The separate Experiments document describes the protocols (split, cross-validation, learning curves, tuning) in full; here they are tied to the chapter's own propositions. Formulas marked [on your sheet] are on your two-page sheet.

Contents

1 Generalisation, the gap, and inductive bias **2**

2 Model capacity: underfitting and overfitting **4**

3 Bias and variance **6**

4 Controlling complexity: regularisation and early stopping **8**

5 Feature and model selection, and the overuse of validation **11**

6 Reliable evaluation: splits, metrics, leakage **13**

7 Diagnosis: learning curves, and Lab 1 **16**

1 Generalisation, the gap, and inductive bias

Empirical risk, population risk, generalisation gap

For a model f and a dataset $\mathcal{D}$ of N examples, the empirical risk is $\widehat{R}_{\mathcal{D}}(f) = \frac{1}{N}\sum_i \ell(y^{(i)}, f(x^{(i)}))$ and the population risk is $R(f) = \mathbb{E}[\ell(Y, f(X))]$ [on your sheet]. For the model $\hat{f}_{\mathcal{D}}$ fitted on $\mathcal{D}$, the *generalisation gap* is

$$G(\hat{f}_{\mathcal{D}}) = R(\hat{f}_{\mathcal{D}}) - \widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}}). \quad \text{[on your sheet]}$$

Both terms refer to the *same* fitted model; only the source of the examples differs. A spam filter with 2% errors on its training emails and 9% on next month's emails has a gap of 0.07.

Purpose. To give a name and a sign to "the fit did not carry over".

Impact. Only $\widehat{R}_{\mathcal{D}}$ is observable. Everything in the chapter is about estimating R, or bounding the gap, without the data that chose the model.

Training risk of a fitted model is optimistic

Let $\hat{f}_{\mathcal{D}}$ minimise $\widehat{R}_{\mathcal{D}}$ over $\mathcal{H}$ and $f^*_{\mathcal{H}}$ minimise R over $\mathcal{H}$. Since $\hat{f}_{\mathcal{D}}$ is the minimiser, $\widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}}) \le \widehat{R}_{\mathcal{D}}(f^*_{\mathcal{H}})$ for every sample. Take expectations over $\mathcal{D}$: $f^*_{\mathcal{H}}$ is fixed, so $\mathbb{E}[\widehat{R}_{\mathcal{D}}(f^*_{\mathcal{H}})] = R(f^*_{\mathcal{H}})$ (an average of i.i.d. losses is unbiased). And $R(\hat{f}_{\mathcal{D}}) \ge R(f^*_{\mathcal{H}})$ pointwise. Therefore

$$\mathbb{E}_{\mathcal{D}}[\widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}})] \;\le\; \min_{f\in\mathcal{H}} R(f) \;\le\; \mathbb{E}_{\mathcal{D}}[R(\hat{f}_{\mathcal{D}})], \qquad \text{so} \qquad \mathbb{E}_{\mathcal{D}}[G(\hat{f}_{\mathcal{D}})] \ge 0. \quad \text{[on your sheet]}$$

The mechanism is that the data play two roles, $\mathcal{D} \to \hat{f}_{\mathcal{D}} \to \widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}})$: they choose the model and then grade it, so the procedure can exploit accidental regularities of $\mathcal{D}$. A second sample $\mathcal{D}'$ would give another $\hat{f}_{\mathcal{D}'}$; that variability is part of learning, not a defect.

Seeing the gap: fit, then test on fresh cases

Setup. Any supervised dataset; a flexible family (a deep tree, a high-degree polynomial).

Held fixed. The family, the training rows.

What we manipulate. Nothing; the point is the two measurements.

What we measure. The risk on the training rows, and the risk on rows never used (held out before fitting, or next month's data).

What the plot shows. Two bars. The held-out bar is higher; the difference is the gap.

How to read it. A gap near zero: training performance transfers. A large gap: the training score was flattering. Low training loss is *necessary* evidence that optimisation worked and *incomplete* evidence that learning worked. That is the entire justification for validation and test sets.

Stable and spurious regularities

A training sample contains regularities of two kinds: those that describe the relationship you want (animal shape predicts "cow"), and those created by how the data were collected (the background predicts "cow" because cows were photographed on grass and camels on sand). Both fit every training example; training performance cannot tell them apart. Dataset composition is therefore *part of the learning method.*

Purpose. To explain how a model can have zero training loss and a large gap: it learned a cue that is perfect in the sample and absent at deployment.

Impact. Spurious cues appear wherever a nuisance variable co-varies with the label in the collection: a network segment or capture time predicting attacks, a scanner type identifying a diagnosis, a customer identifier predicting churn.

The shortcut probe

Setup. A classifier trained on the collection (cows on grass, camels on sand).

Held fixed. The fitted model.

What we manipulate. The nuisance factor alone: present the model with a cow on sand, a camel on grass, or, in intrusion detection, flows from a new network segment or a new week; or shuffle the labels and retrain; or delete the metadata columns and retrain.

What we measure. Accuracy on the probe cases versus on the original test split.

What the plot shows. A table: original split high; probe cases much lower; label-shuffled model at chance; metadata-free model close to the original if the shortcut was not used, far below it if it was.

How to read it. A drop on the probe is a shortcut. Shuffled labels must collapse to chance (if they do not, something leaks the label). Removing metadata must not collapse performance (if it does, the model was reading identifiers, not behaviour).

Weakening shortcuts in practice

Vary nuisance factors at collection time (several cameras, sites, seasons, devices); augment only with label-preserving, realistic transformations (flip a cat photo: fine; flip the digit 6: it becomes a 9); and evaluate under deployment-like conditions, because a random split hides a shortcut that is present in every part of the collection. In the wolves-and-snow story the probe was a wolf on grass; in security it is a new segment or a new month.

Inductive bias

A finite sample is compatible with infinitely many functions that agree on the sample and differ elsewhere. Learning therefore needs *assumptions that make some solutions more plausible than others off the sample*: the inductive bias. It enters through (1) the data and representation (which cues exist), (2) the hypothesis space (which relationships can be expressed), (3) regularisation (which fits are preferred), and (4) the optimisation algorithm (which of many minimisers is reached).

Purpose. To make clear that generalisation is never a property of the data alone; it succeeds when the assumptions remain true at deployment.

Impact. Every technique of this chapter chooses, strengthens or checks one of the four entries. "Bias" here means a preference among solutions; it is broader than the statistical bias of Section 3.

Inductive bias versus statistical bias

	Inductive bias	Statistical bias $\mathrm{Bias}(x)$
What it is	a preference among functions off the sample	average prediction minus truth, over training sets
Where it lives	data, family, regulariser, optimiser	a number at each x
Good or bad	necessary; wrong if it fails at deployment	a cost, traded against variance

Key insight. The sample chooses the model and then grades it; that is why the training score is optimistic. What decides the next case is not the data but the assumptions (inductive bias), and whether they still hold where the model is used.

How the professor asks it

Q. *Prove that the expected generalisation gap of an empirical risk minimiser is nonnegative, and say exactly where the assumption "$\hat{f}_{\mathcal{D}}$ minimises the empirical risk" is used.*
A. Claim: $\mathbb{E}_{\mathcal{D}}[\widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}})] \leq \mathbb{E}_{\mathcal{D}}[R(\hat{f}_{\mathcal{D}})]$. Let $f^*_{\mathcal{H}}$ be the population minimiser in $\mathcal{H}$. Minimisation is used once: it gives $\widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}}) \leq \widehat{R}_{\mathcal{D}}(f^*_{\mathcal{H}})$ on every sample. Taking expectations, the right side equals $R(f^*_{\mathcal{H}})$ because $f^*_{\mathcal{H}}$ is fixed and the loss average is unbiased for a fixed function. Finally $R(f^*_{\mathcal{H}}) \leq R(\hat{f}_{\mathcal{D}})$ pointwise because $f^*_{\mathcal{H}}$ is best in the family. Chain the three facts. Consequence: a model's own training score is not evidence about new cases.

Q. *A model reaches 100% on an image collection in which all cows stand on grass and all camels on sand. Explain why this number is uninformative and design the experiment that would reveal the problem.*
A. Two cues, shape and background, predict every training label, and training performance cannot separate them: both give zero loss. Only inputs where the cues disagree can. Experiment: collect or construct cows on sand and camels on grass (hold the animal fixed, manipulate the background), measure accuracy on them; a collapse shows the model used the background. Complement with a label-shuffle control (must fall to chance) and with a grouped or site-wise split (hold out whole locations). The fix is at collection time: vary the background, or augment with realistic label-preserving changes.

2 Model capacity: underfitting and overfitting

Capacity and nested families

The capacity of $\mathcal{H} = \{f_\theta\}$ is how varied its functions can be. It is set by structure (degree, depth, width), constraints, regularisation, the optimiser and the data, not by the parameter count alone. Families are *nested* when $\mathcal{H}_1 \subseteq \mathcal{H}_2 \subseteq \ldots$: every function of a smaller family is available in a larger one. Typical knobs: polynomial degree, tree depth, number of units, and k in kNN (capacity grows as k *shrinks*).

Purpose. To have one word for "how much the family can bend to the sample".

Impact. Training error can only fall with capacity: $\min_{\mathcal{H}_{m+1}} \widehat{R}_{\text{train}} \leq \min_{\mathcal{H}_m} \widehat{R}_{\text{train}}$ [on your sheet], because the larger family can reuse the smaller one's minimiser. Nothing of the sort holds for the population risk.

Underfitting and overfitting

An *underfitting* model is too restricted to capture important structure: high training *and* high validation error. An *overfitting* model is too sensitive to the particular sample: very low training error, substantially higher validation error. The distinction is not whether the model is complicated in isolation but whether its effective flexibility matches the available evidence: a family unstable on 50 points may be well constrained by 50,000.

The validation curve: sweep capacity

Setup. Training rows, validation rows (or folds), a nested family with a knob.

Held fixed. Rows, preprocessing (fitted on training rows), metric.

What we manipulate. Degree $0, 1, 2, \ldots$; or depth; or $k = 1, 3, 5, \ldots$ (plot against $1/k$ so capacity increases to the right).

What we measure. Training and validation risk at each knob value.

What the plot shows. Training risk decreasing monotonically; validation risk falling then rising: the U.

How to read it. Left of the minimum: both high, underfitting. Right: training low and validation rising, overfitting. The minimum is the capacity the data can support; with more data the minimum moves right. For kNN remember the direction: small k means small adaptive neighbourhoods (low training error, high sensitivity), large k averages broadly.

Three polynomials on four points, judged on a fifth

Points $(0,1),(1,3),(2,2),(3,4)$ fitted by degree 0, 1, 3 give training risks 1.25, 0.45, 0. A validation point $(4,5)$ arrives. Degree 0 (constant 2.5) predicts 2.5, error 6.25. Degree 1 ($0.8x+1.3$) predicts 4.5, error 0.25. Degree 3 (the interpolating cubic $1+5.5x-4.5x^2+x^3$, which passes through all four points) predicts $1+22-72+64=15$, error 100. The constant underfits; the cubic has the best training score and the worst validation score, because it bent itself around the noise of four points; the line, worse than the cubic on the training points, is the one to deploy.

Capacity mistakes

- Treating zero training error as an achievement. With enough capacity it is guaranteed (enough flexibility fits anything, Chapter 2); it is a warning to look at validation error.
- Reading the kNN direction backwards: *decreasing* k increases capacity.
- Judging capacity by parameter count: a heavily regularised million-parameter model can have less effective capacity than an unregularised cubic (Section 4).
- Comparing families by training error (always favours the larger family, proof above).

Underfitting versus overfitting

	Training error	Validation error	Fix
Underfitting	high	high, close to training	more capacity, better features, less regularisation
Overfitting	low	much higher than training	regularise, reduce capacity, more data, ensembles
Well matched	low	slightly above training	stop

Key insight. Capacity is what the training error rewards and the validation error punishes beyond a point; the U is the signature of a family that can fit both the signal and the sample's accidents.

How the professor asks it

Q. *Why does training error never increase when you go from polynomial degree 2 to degree 3, and why does this not imply that degree 3 generalises better?*
A. Degree-2 polynomials are degree-3 polynomials with the cubic coefficient zero, so the degree-2 minimiser is a candidate in the degree-3 family and the degree-3 minimum is at most its value: nested families, training error nonincreasing. The population risk has no such monotonicity: the extra coefficient can fit stable structure or accidental variation, and on four points the cubic fits all the noise (training risk 0, validation error 100 in the example). Which happened is decided by validation data, never by the training score.

3 Bias and variance

Bias and variance of a learning procedure

At a fixed input x, with $f^*(x) = \mathbb{E}[Y \mid X = x]$ and $\bar{f}(x) = \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)]$ the average prediction across training sets,

$$\text{Bias}(x) = \bar{f}(x) - f^*(x), \qquad \text{Var}(x) = \mathbb{E}_{\mathcal{D}}[(\hat{f}_{\mathcal{D}}(x) - \bar{f}(x))^2]. \quad \text{[on your sheet]}$$

Fix one house; train a thousand times on a thousand training sets; record the thousand predicted prices. Bias: is their average off from the true mean price in a consistent direction? Variance: how widely do they scatter around their own average? A model that always says "1 million" has zero variance and possibly large bias; a very flexible model may have tiny bias and huge variance.

Purpose. To turn "sensitive to the sample" into a number and separate it from "systematically wrong".

Impact. Restrictive families and poor features create bias, which more data does not remove. High capacity, small samples, irrelevant variables and unstable feature selection create variance, which more representative data reduces.

The bias–variance decomposition, with proof

Assume $Y = f^*(X) + \varepsilon$, $\mathbb{E}[\varepsilon \mid X] = 0$, $\text{Var}(\varepsilon \mid X = x) = \sigma^2(x)$, and the training sample independent of the test pair. Then at every x,

$$\mathbb{E}_{\mathcal{D},Y|x}[(Y - \hat{f}_{\mathcal{D}}(x))^2] = \sigma^2(x) + \text{Bias}(x)^2 + \text{Var}(x). \quad \text{[on your sheet]}$$

Step 1 (split off the noise). $Y - \hat{f}_{\mathcal{D}}(x) = \varepsilon + (f^*(x) - \hat{f}_{\mathcal{D}}(x))$. Square: $\varepsilon^2 + 2\varepsilon(f^* - \hat{f}_{\mathcal{D}}) + (f^* - \hat{f}_{\mathcal{D}})^2$. $\mathbb{E}[\varepsilon^2 \mid x] = \sigma^2(x)$. In the cross term ε is independent of $\mathcal{D}$ and has mean zero, so $\mathbb{E}[\varepsilon(f^* - \hat{f}_{\mathcal{D}})] = \mathbb{E}[\varepsilon]\,\mathbb{E}_{\mathcal{D}}[f^* - \hat{f}_{\mathcal{D}}] = 0$.
Step 2 (split around the average prediction). $f^* - \hat{f}_{\mathcal{D}} = (f^* - \bar{f}) + (\bar{f} - \hat{f}_{\mathcal{D}})$. The first bracket is the constant $-\text{Bias}(x)$, whose square is Bias^2; the second has $\mathbb{E}_{\mathcal{D}}$-mean zero by definition of $\bar{f}$, so the cross term $2(f^* - \bar{f})\mathbb{E}_{\mathcal{D}}[\bar{f} - \hat{f}_{\mathcal{D}}]$ vanishes; its square averages to $\text{Var}(x)$.
Twice the same move: add and subtract a mean, square, and the cross term dies because a mean-zero quantity multiplies a constant. The three pieces: irreducible noise (even f^* makes it), systematic error (repeating never averages it away), sample sensitivity (a different sample would change it).

Measuring bias and variance by repeated sampling

Setup. A simulated world with known f^* and noise; a procedure (family plus fitting rule plus N).

Held fixed. f^*, σ, N, a grid of test inputs.

What we manipulate. The training set: draw it 500 times and refit each time. In a second sweep, vary capacity (degree, k) and in a third, vary N.

What we measure. At each test x: the mean and spread of the 500 predictions; Bias^2 and Var averaged over the grid; the measured test error for comparison.

What the plot shows. Against capacity: bias^2 falling, variance rising, their sum plus σ^2 equal to the test error and U-shaped. Against N: variance falling, bias flat.

How to read it. Capacity trades bias for variance (a useful principle, not a law). Data buys variance only. The minimum of the U moves to higher capacity as N grows.

Bias and variance computed exactly

No input: $Y = \mu + \varepsilon$, $\mathrm{Var}(\varepsilon) = \sigma^2$, N draws. *Sample mean*: $\mathbb{E}[\hat{f}] = \mu$, bias 0, variance σ^2/N. *Ignore the data, predict 0*: bias $-\mu$, variance 0. *Shrunk mean* ($\frac{1}{2}$ of the sample mean): bias $-\mu/2$, variance $\sigma^2/(4N)$. With $\sigma^2 = 1$, $N = 4$: bias^2+variance is 0.25, 1, 0.3125 for $\mu = 1$ and 0.25, 0.04, 0.0725 for $\mu = 0.2$. When the truth is far from zero the unbiased procedure wins; when it is near zero both biased procedures beat it by trading a little systematic error for a large variance reduction. Regularisation is exactly this trade, made tunable.

Three sources of excess error, and double descent

Relative to the Bayes risk, a learned model's extra risk is *approximation error* (the family cannot express a good enough function), *estimation error* (finite data do not identify the best member) and *optimisation error* (the algorithm does not reach a good solution of its own objective). Capacity lowers the first and raises the second. *Double descent*: in modern overparameterised models the test error can peak at the interpolation threshold (parameters $\approx N$) and fall again beyond it [on your sheet]; it depends on regularisation, optimiser, data geometry and which interpolating solution is selected, and an analogue occurs over training time. The classical U remains the first model; validation behaviour must be examined, not assumed.

Bias–variance mistakes

- "The variance of this trained model." Variance is across training sets; one model is one draw.
- "Doubling the data halves the bias." It halves the variance in the example; bias comes from the family and does not move.
- "Moderate complexity gives low bias and low variance." At the bottom of the U both are positive, just small; zero bias and zero variance would mean the procedure returns f^* regardless of the sample.
- Using the decomposition for 0–1 loss or without independence of training and test noise.
- 1-NN: small bias, large variance (one noisy neighbour decides). Students often say the opposite.

Noise versus bias versus variance

	What it is	What changes it	Quick example
$\sigma^2(x)$	randomness of Y that x does not explain	better features only	two identical houses, different prices
Bias^2	average prediction off the truth	family, features, regularisation (up)	constant model on a curve
Var	spread across training sets	capacity (up), N (down), averaging (down)	1-NN, interpolating cubic

Key insight. Add and subtract the true function, then add and subtract the average prediction: squared error splits into noise, bias^2 and variance, and the three respond to different levers (features, family, data).

How the professor asks it

Q. *A procedure has* $\mathrm{Bias}(x) = 0.3$, $\mathrm{Var}(x) = 0.05$ *and noise* $\sigma^2 = 0.1$ *at* x. *Compute its expected squared error, state which part more data could reduce, and which part a richer family could reduce.*

A. $0.1 + 0.3^2 + 0.05 = 0.1 + 0.09 + 0.05 = 0.24$. More data acts on the variance 0.05 (it shrinks roughly like $1/N$ for stable procedures); a richer family acts on the bias 0.09 (it can move $\bar{f}$ closer to f^*) at the price of raising the variance; nothing acts on the noise 0.1 except new information in the features.

Q. *Can a learning procedure have zero bias and zero variance? Justify.*
A. Only if $\hat{f}_{\mathcal{D}}(x) = f^*(x)$ for every training set, i.e. the procedure returns the truth regardless of the sample, which means it already knew the answer. Any procedure that actually uses the data has either sample dependence (variance) or a restriction that keeps it from f^* (bias), usually both.

4 Controlling complexity: regularisation and early stopping

Regularised empirical risk minimisation

With a penalty $\Omega : \Theta \to [0, \infty)$ and strength $\lambda \geq 0$,

$$\hat{\theta}_\lambda \in \arg\min_\theta \left[\widehat{R}_{\text{train}}(\theta) + \lambda\Omega(\theta)\right], \quad \text{[on your sheet]}$$

$$\Omega_2 = \|\theta\|_2^2 = \sum_j \theta_j^2 \text{ (ridge)}, \qquad \Omega_1 = \|\theta\|_1 = \sum_j |\theta_j| \text{ (lasso)}. \quad \text{[on your sheet]}$$

"Fit the data *and* keep the parameters small." λ says how much a unit of penalty costs in units of training loss: $\lambda = 0$ is plain ERM; large λ makes any sizeable coefficient too expensive. With coefficients $(3, -4)$: $\Omega_2 = 25$, $\Omega_1 = 7$.

Purpose. To reduce effective capacity without removing parameters: the model keeps all its numbers but is no longer free to use them.

Impact. Larger λ means a smaller effective family (next box), more bias from shrinkage, less variance. Other dials with the same purpose: restricting the family, increasing k, selecting features, stopping early, collecting more relevant data.

A penalty is a soft constraint, and the families are nested

Let $\hat{\theta}_\lambda$ solve the penalised problem and put $c = \Omega(\hat{\theta}_\lambda)$. Claim: $\hat{\theta}_\lambda$ also solves $\min_\theta \widehat{R}_{\text{train}}(\theta)$ subject to $\Omega(\theta) \leq c$ [on your sheet]. Take any feasible θ ($\Omega(\theta) \leq c$). Optimality of $\hat{\theta}_\lambda$ for the penalised problem gives $\widehat{R}(\hat{\theta}_\lambda) + \lambda c \leq \widehat{R}(\theta) + \lambda\Omega(\theta) \leq \widehat{R}(\theta) + \lambda c$, hence $\widehat{R}(\hat{\theta}_\lambda) \leq \widehat{R}(\theta)$: no feasible point beats $\hat{\theta}_\lambda$, which is itself feasible. The constrained families $\mathcal{H}_c = \{f_\theta : \Omega(\theta) \leq c\}$ are nested in c, and $\lambda \uparrow$ corresponds to $c \downarrow$ (the exact map depends on the problem). Consequences: (i) regularised ERM is ERM over a smaller effective family, so capacity is about which functions can win the fitting contest, not how many numbers are stored; (ii) training risk is minimised at $\lambda = 0$ (nested families), so λ cannot be chosen on training data.

Ridge in one variable: smooth shrinkage

Model $f_w(x) = wx$, objective $J(w) = \frac{1}{N}\sum_i (y^{(i)} - wx^{(i)})^2 + \lambda w^2$, a convex quadratic. $J'(w) = -\frac{2}{N}\sum_i x^{(i)}(y^{(i)} - wx^{(i)}) + 2\lambda w = 0$ gives $w(\sum_i (x^{(i)})^2 + N\lambda) = \sum_i x^{(i)} y^{(i)}$, so

$$\hat{w}_\lambda = \frac{\sum_i x^{(i)} y^{(i)}}{\sum_i (x^{(i)})^2 + N\lambda} = \hat{w}_{\text{LS}} \cdot \frac{\sum_i (x^{(i)})^2}{\sum_i (x^{(i)})^2 + N\lambda}, \quad \text{[on your sheet]}$$

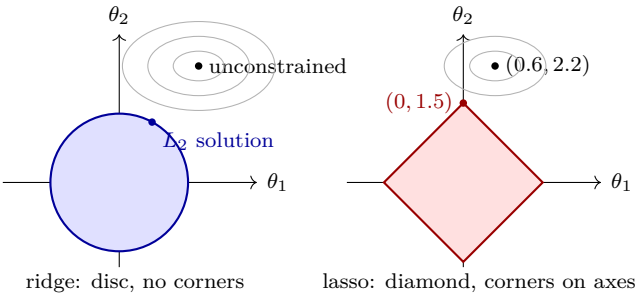
the least-squares slope times a factor in $(0, 1]$: shrunk toward 0, smoothly, never exactly 0 for finite λ. With $\sum x^2 = 14$, $\sum xy = 19$, $N = 4$: $19/14 = 1.357$ at $\lambda = 0$, $19/18 = 1.056$ at $\lambda = 1$, $19/54 = 0.352$ at $\lambda = 10$. As $\lambda \to \infty$ all penalised coefficients go to 0 and the model becomes the intercept alone.

Lasso in one variable: soft thresholding and exact zeros

Minimise $\frac{1}{2}(w-z)^2+\lambda|w|$ over $w\in\mathbb{R}$, where z is where the unpenalised fit would put the coefficient. Case $z>0$. For $w\geq 0$ the objective is $\frac{1}{2}(w-z)^2+\lambda w$, a parabola with unconstrained minimiser $w=z-\lambda$: if $z>\lambda$ this is positive and is the minimum on $[0,\infty)$; if $z\leq\lambda$ the parabola increases on $[0,\infty)$ and the minimum is at $w=0$. For $w<0$ the objective is $\frac{1}{2}(w-z)^2-\lambda w$, whose unconstrained minimiser $z+\lambda$ is positive, so it decreases toward 0 and no negative w beats $w=0$. Hence $\hat{w}=\max(z-\lambda,0)$ for $z>0$; by symmetry

$$\hat{w}=\operatorname{sign}(z)\max(|z|-\lambda,0), \quad \text{[on your sheet]}$$

and $\hat{w}=0$ exactly whenever $|z|\leq\lambda$. With $\lambda=0.5$: $z=2.2\to 1.7$; $z=0.4\to 0$; $z=-3\to -2.5$. With $\lambda=1.5$ and $z=1.2$: 0. Geometrically the L_1 ball $\|\theta\|_1\leq c$ is a diamond with corners on the axes and the optimum often lands on a corner where coordinates are exactly zero (the slides' $(0.6,2.2)\to(0,1.5)$: feature 1 dropped); the L_2 ball is a disc without corners, so ridge shrinks everything and keeps everything. L_1 therefore performs *embedded feature selection*; with correlated features the selected subset can be unstable, and in more than one dimension the lasso solution is not obtained by coordinate-wise soft thresholding, which is why the result is stated in one variable.



The regularisation path: sweep λ

Setup. Standardised features (fitted on training rows), ridge and lasso, training and validation rows.

Held fixed. Rows, features, metric; the intercept unpenalised.

What we manipulate. λ on a logarithmic grid $10^{-3},\ldots,10^{2}$.

What we measure. Every coefficient at every λ (the *path*), plus training and validation risk.

What the plot shows. Coefficient paths against $\log\lambda$: ridge paths shrink smoothly toward zero and never reach it; lasso paths hit zero one after another, so the number of nonzero coefficients steps down. Below them, the validation U.

How to read it. The minimum of the U gives $\hat{\lambda}$. On the lasso path, the order in which coefficients die is a feature ranking; with two near-identical features ridge splits the weight equally while lasso keeps one and may switch which one between folds (instability). Without standardisation a feature in millimetres is penalised a thousand times more than the same feature in metres, and the path is meaningless.

Credit scoring with 300 candidate variables

A bank has 300 features for 5,000 loans. Ridge keeps all 300 with small weights: good prediction, no story. Lasso at the validation-chosen λ keeps 18, which the risk team can audit and which are cheap to collect at application time. Among two near-duplicate income variables lasso kept one; which one changes between folds, so the team reports the pair, not the individual. The L_1 choice was driven by the belief that few variables matter and the need to know which; L_2 is the

choice when many variables each contribute a little.

Early stopping

Iterative optimisation produces $\hat{\theta}_0, \hat{\theta}_1, \ldots$. Early iterations learn strong recurring patterns; later ones fit exceptions and noise. Early stopping keeps the checkpoint with the smallest validation risk, $t^* = \arg\min_t \widehat{R}_{\text{val}}(\hat{\theta}_t)$ [on your sheet], with a *patience* p: continue p epochs without improvement before stopping, keeping the best checkpoint seen.

Purpose. Regularisation for free in any iterative method: training time controls effective capacity while the parameter count stays fixed.

Impact. The stopping epoch is a hyperparameter chosen on validation. Patience protects against stopping on a wobble; it can still miss a later decrease under epoch-wise double descent.

Early stopping: the training curve over epochs

Setup. An iterative learner, training and validation rows.

Held fixed. Everything except the number of epochs.

What we manipulate. The epoch t, by training on and saving a checkpoint each epoch.

What we measure. Training and validation loss after each epoch.

What the plot shows. Training loss falls throughout; validation loss falls, bottoms at t^*, rises.

How to read it. Roll back to $\hat{\theta}_{t^*}$, never keep the last epoch. Stopping on the test loss is tuning on the test set.

Regularisation mistakes

- Choosing λ on training data: training risk is minimised at $\lambda = 0$ (nested families), so the answer is always "no regularisation".
- Penalising unstandardised features, or penalising the intercept.
- Searching λ on a linear grid: the useful range spans orders of magnitude.
- Expecting ridge to select features (it never produces exact zeros) or lasso to be stable under correlated features (it is not).
- Keeping the last checkpoint instead of the best one; stopping without patience on a noisy validation curve.

L_2 versus L_1 versus early stopping versus smaller family

Dial	What it manipulates	Effect on coefficients	Use when
L_2 (ridge)	size of $\|\theta\|_2$	smooth shrinkage, none exactly zero	many features each contribute a little
L_1 (lasso)	size of $\|\theta\|_1$	soft threshold, exact zeros	few features matter and you want to know which
early stopping	number of iterations	the path stops before fitting noise	any iterative method; free
smaller family	degree, depth, width, larger k	fewer functions available	structure is known to be simple

Key insight. A penalty does not remove parameters; it removes functions from the contest. Ridge shrinks every coefficient by a factor, lasso subtracts λ and stops at zero, early stopping limits how far the optimiser travels; all three are chosen on validation because training error would set them to nothing.

How the professor asks it

Q. *Derive the one-variable ridge solution and explain in words why it can never set the coefficient exactly to zero, while the lasso can.*
A. Minimise $\frac{1}{N}\sum(y-wx)^2+\lambda w^2$: the derivative $-\frac{2}{N}\sum x(y-wx)+2\lambda w=0$ gives $\hat{w}_\lambda=\sum xy/(\sum x^2+N\lambda)$, the least-squares slope times $\sum x^2/(\sum x^2+N\lambda)\in(0,1]$. The factor is strictly positive for finite λ, so the coefficient keeps the sign and magnitude order of the unpenalised one, only scaled. The L_2 penalty's derivative $2\lambda w$ vanishes at $w=0$, so there is no force holding the coefficient at zero. The L_1 penalty has a kink at zero: its subgradient there is the whole interval $[-\lambda,\lambda]$, so whenever the data's pull $|z|$ is at most λ the kink absorbs it and $\hat{w}=0$ exactly (soft threshold $\operatorname{sign}(z)\max(|z|-\lambda,0)$).

Q. *Two identical features are included in a ridge model and in a lasso model. What does each do with them, and why does this matter for interpreting a lasso's selected subset?*
A. Ridge splits the weight equally (any other split has the same fit and a larger $\|\theta\|_2^2$, since a^2+b^2 is minimised at $a=b$ for fixed $a+b$). Lasso is indifferent among all splits with the same sum ($|a|+|b|$ is constant for same-sign splits), so it may keep either one and drop the other, and which one survives can change between folds or samples. A lasso's selected subset is therefore an unstable ranking under correlation; report groups of correlated features, or use the path across folds.

5 Feature and model selection, and the overuse of validation

Feature selection as a model choice

Choosing which inputs to use changes the representation. It can remove irrelevant variables (which increase estimation difficulty), reduce redundancy, improve interpretability, and exclude variables unavailable at decision time. It is a data-dependent choice and must be evaluated like any other: on rows that took no part in it.

Purpose. Fewer, relevant inputs mean less variance and a model you can explain.

Impact. With d features there are 2^d subsets [on your sheet]: exhaustive search costs 2^d fits, and $2^{30}=1{,}073{,}741{,}824$. Practical searches are filters and greedy procedures, each blind to something.

Filter versus forward versus backward versus embedded selection

Method	What it does	Cost	Blind to
Correlation filter	scores each variable alone, $s_j=\lvert\widehat{\mathrm{Corr}}(X_j,Y)\rvert$, keeps the top ones	d scores, no fits	nonlinear relations, interactions, redundancy between kept features
Forward selection	start empty, add the feature with the largest validation gain, repeat	about $d(d+1)/2$ fits [on your sheet]	combinations blocked by an early greedy choice
Backward elimination	start full, remove the least useful, repeat	same order; needs the full model to be fittable	same; impossible when $d>N$
Embedded (L_1)	the penalty zeroes coefficients during fitting	one path	unstable under correlated features

Useless alone, perfect together

X_1, X_2 independent fair ±1 coins and $Y=X_1X_2$. Each feature has correlation 0 with Y (knowing X_1 says nothing about the sign of the product), so a correlation filter discards both; together they determine Y exactly. Filters look at one variable at a time; interactions are invisible to

them. Weak marginal correlation never proves a feature useless jointly with others.

Feature selection inside versus outside the folds

Setup. 300 rows, 5,000 features, labels generated independently of the features (no signal).

Held fixed. The data, 10-fold cross-validation of logistic regression on 20 features.

What we manipulate. Where the selection step runs: (a) rank features on all 300 rows, keep the top 20, then cross-validate; (b) inside each fold, rank on that fold's 270 training rows only.

What we measure. Cross-validation accuracy in both arrangements.

What the plot shows. (a) reports high accuracy (often above 90%); (b) reports about 50%.

How to read it. In (a) the validation rows helped choose the representation they are scored on: leakage. All data-dependent scoring and subset search belong *inside* the training pipeline and are repeated within every fold [on your sheet].

The three subsets and the best-of-many problem

Let h be an entire configuration (features, family, λ, k, threshold). Training fits $\hat{f}_h$; validation selects $\hat{h} \in \arg\min_{h \in \mathcal{C}} \widehat{R}_{\text{val}}(\hat{f}_h)$ [on your sheet]; the test set stays outside the loop. Each validation score is honest for its own candidate; the *minimum* over candidates is optimistic.

The best of many validation scores is optimistic

If each $\widehat{R}_{\text{val}}(\hat{f}_h)$ is unbiased for $R(\hat{f}_h)$ (models fitted without the validation rows), then for every h_0, $\min_h \widehat{R}_{\text{val}}(\hat{f}_h) \le \widehat{R}_{\text{val}}(\hat{f}_{h_0})$ pointwise; take expectations and then the minimum over h_0 on the right:

$$\mathbb{E}\Big[\min_{h \in \mathcal{C}} \widehat{R}_{\text{val}}(\hat{f}_h)\Big] \le \min_{h \in \mathcal{C}} R(\hat{f}_h), \quad \text{[on your sheet]}$$

strict as soon as the estimates are noisy and no single h wins in every sample. Try a hundred configurations and report the lowest score: that is the minimum of a hundred noisy numbers, pulled downward. Every reported validation result also steers the next design choice, so development gradually adapts to the validation set. The test set protects against this accumulated adaptation, and only if its result is not then used to continue development, at which point it has become another validation set. For the magnitude ($\sigma/\sqrt{\pi}$ for two equal candidates, $\sigma\sqrt{2\ln n}$ for n) see the Experiments document.

K-fold cross-validation for configurations

Partition the development data into K folds; for configuration h and fold j fit $\hat{f}_h^{(-j)}$ without fold j and evaluate on it; $\widehat{R}_{\text{CV}}(h) = \frac{1}{K}\sum_j \widehat{R}_{\text{val}}^{(j)}(\hat{f}_h^{(-j)})$, $\hat{h} \in \arg\min_h \widehat{R}_{\text{CV}}(h)$ [on your sheet]. Each observation validates once and trains $K-1$ times. It uses limited data efficiently at K fits per configuration; the estimate still varies with the sample; random folds assume exchangeable rows (grouped or temporal data need adapted folds); and it does not replace a final untouched test set.

Selection mistakes

- Scoring or selecting features on all rows, then cross-validating: leakage, inflated estimate.
- Trusting a correlation filter to find interacting features (the XOR example).
- Reporting the best validation (or cross-validation) score as the expected performance.
- Continuing to develop after looking at the test score.

- Using random folds on grouped or temporal data.

Key insight. Selection is part of training: whatever looks at the labels to choose features, hyperparameters or thresholds must be redone inside each fold, and the winner's score is optimistic by construction, which is why one untouched test set exists.

How the professor asks it

Q. *Explain why the minimum of 100 honest validation estimates is not an honest estimate of the winner's risk. Give the one-line inequality.*
A. Each estimate scatters around its own true risk; picking the minimum picks, in part, the luckiest scatter. Formally $\min_h \widehat{R}_{\text{val}}(\hat{f}_h) \le \widehat{R}_{\text{val}}(\hat{f}_{h_0})$ for any fixed h_0, so $\mathbb{E}[\min_h \widehat{R}_{\text{val}}(\hat{f}_h)] \le \min_{h_0} R(\hat{f}_{h_0})$: the expected best score lies below the best true risk, with strict inequality whenever the estimates are noisy and no single configuration always wins. The winner's true risk is at least $\min_h R(\hat{f}_h)$, so the reported score underestimates it. Only rows that took no part in the selection can estimate it honestly.

Q. *Why must feature selection be repeated inside every cross-validation fold rather than done once before cross-validation?*
A. Because selection reads the labels. If it reads the labels of the rows that will later be a validation fold, that fold's labels have influenced the representation on which the fold is then scored, so the fold is no longer unseen; on label-free data this produces high cross-validation accuracy from nothing. Repeating the selection inside each fold, on the training portion only, restores the condition that the validation rows played no part in anything that was chosen.

6 Reliable evaluation: splits, metrics, leakage

The split must match deployment

Ask first: what kind of *new* case will the model face? Then hold out exactly that kind. *Random*: each observation assigned independently (credible only when deployment cases are independent draws from the same population). *Stratified*: random but preserving class proportions in every subset, stabilising rare-class counts; it must not override dependence (records from one device must not be scattered to match proportions). *Grouped*: every group (user, device, patient, hospital) entirely in one subset, so the learner cannot use identity and is evaluated on unseen groups. *Temporal*: train on the past, validate and test on later observations, preserving the direction of prediction and revealing deterioration as behaviour or attacks change. [on your sheet]

Purpose. To make the held-out number describe what will happen at deployment.

Impact. Random, grouped and temporal splits of the same data give different numbers; only the one that mirrors deployment is predictive.

Three protocols, one model

Setup. Flows from 50 devices over 6 months; one model.

Held fixed. The model, the features, the metric.

What we manipulate. The split rule: random; grouped by device (hold out 10 devices); temporal (train months 1–4, test months 5–6, during which a new attack tool appeared).

What we measure. Test accuracy under each.

What the plot shows. Random 99.5% (the model learned "device 17 is the compromised laptop"); grouped 96% (it must recognise behaviour); temporal 90% (new tool).

How to read it. Only the third number describes next month. "Will this customer churn next

quarter" is temporal (features before, outcome after); "does this model work on a hospital it has never seen" is grouped by hospital.

Metrics from the confusion matrix

With the rare class (attacks) positive, count TP, FN, FP, TN. [on your sheet]

$$\text{Recall} = \frac{\text{TP}}{\text{TP}+\text{FN}}, \qquad \text{Precision} = \frac{\text{TP}}{\text{TP}+\text{FP}}, \qquad \text{Specificity} = \frac{\text{TN}}{\text{TN}+\text{FP}},$$
$$\text{BalancedAcc} = \frac{\text{Recall}+\text{Specificity}}{2}, \qquad F_1 = \frac{2PR}{P+R}.$$

Recall: of the real attacks, what fraction did we catch? Precision: of the alarms, what fraction were real? Specificity: of the benign flows, what fraction were left alone? Balanced accuracy: average of the two per-class rates, immune to imbalance. F_1: harmonic mean of precision and recall, high only when both are, ignores true negatives.

Purpose. Accuracy hides minority failure: with a positive fraction π, the classifier that always says "negative" has accuracy $1-\pi$ and recall 0 [on your sheet]; at $\pi = 0.01$ that is 99% accuracy and no attack ever detected (its balanced accuracy, 0.5, is the number that exposes it).

Impact. The metric and the threshold must reflect the costs of missed attacks, false alarms and alert volume, exactly as the cost-sensitive threshold of Chapter 2.

A detector on 10,000 flows with 50 attacks

TP $= 40$, FN $= 10$, FP $= 90$, TN $= 9860$. Accuracy $9900/10000 = 0.990$; recall $40/50 = 0.80$; precision $40/130 = 0.31$; specificity $9860/9950 = 0.991$; balanced accuracy $(0.80 + 0.991)/2 = 0.895$; $F_1 = 2(0.31)(0.80)/(0.31 + 0.80) = 0.44$. Accuracy says 99%; precision says two alarms in three are false. Which matters depends on whether an analyst must investigate each alarm. Another case: TP $= 30$, FN $= 70$, FP $= 0$ gives precision 1.0, recall 0.3, $F_1 = 0.6/1.3 \approx 0.46$.

F_1 sits between the worse rate and the average

For $P, R \in (0, 1]$: $\min(P, R) \le F_1 \le \frac{P+R}{2}$, with equality on the right iff $P = R$. Lower bound: if $P \le R$ then $F_1 = P \cdot \frac{2R}{P+R} \ge P$ since $2R \ge P + R$. Upper bound: $\frac{P+R}{2} - \frac{2PR}{P+R} = \frac{(P+R)^2-4PR}{2(P+R)} = \frac{(P-R)^2}{2(P+R)} \ge 0$. So F_1 is dragged toward the *worse* of the two: 0.31 and 0.80 give 0.44, far below the average 0.56. That is the point of a harmonic mean: a great recall cannot hide a poor precision.

Threshold sweep on validation

Setup. A probabilistic classifier, validation rows, costs $C_{\text{FP}}, C_{\text{FN}}$.

Held fixed. The fitted model, the validation rows.

What we manipulate. The decision threshold t from 0 to 1.

What we measure. Recall, precision, false alarms per 100,000 benign flows, expected cost at each t.

What the plot shows. Recall falling and precision rising with t; expected cost U-shaped with minimum near the cost threshold $t = C_{\text{FP}}/(C_{\text{FP}} + C_{\text{FN}})$ when the probabilities are calibrated.

How to read it. The threshold is a hyperparameter: choose it on validation, report on test. Lab 1 does exactly this, reporting test recall, false-positive rate and false alarms per 100,000 benign flows.

Leakage

Leakage is any use, during development, of information that will not be available when a new decision is made [on your sheet]: duplicates or near-duplicates in several subsets; the same device, session or patient in training and test; predictors created after the outcome or after a human investigation ("ticket closed as intrusion", "number of analyst comments"); preprocessing fitted with validation or test observations; hyperparameters repeatedly chosen on test results. It creates an unrealistically favourable estimate.

Purpose. A checklist of the ways tomorrow's information gets into today's model.

Impact. The fix for preprocessing: the training set estimates $\hat{\mu}_j = \frac{1}{N_{\text{tr}}} \sum_{i \in \text{tr}} x_j^{(i)}$ and $\hat{\sigma}_j^2$, and the *same* values are reused unchanged on validation, test and deployment [on your sheet]; likewise imputation, feature selection, dimensionality reduction, resampling; in cross-validation each fold fits them on its own training portion. Leaking pipeline: all data $\rightarrow$ estimate $\mu, \sigma \rightarrow$ split. Valid pipeline: split $\rightarrow$ estimate on training $\rightarrow$ reuse.

Evaluation mistakes

- Random split for future events or new entities; stratifying across groups.
- Accuracy on imbalanced data; a metric chosen after seeing the results.
- A feature that predicts perfectly because it was recorded after the outcome.
- Scaler, imputer or selector fitted on all rows; a time series shuffled before splitting.
- Threshold tuned on training or test rows.

Random versus stratified versus grouped versus temporal

Split	Deployment question it answers	What it holds out	Fails when
random	new independent cases from the same population	rows	rows are dependent (groups, time)
stratified	same, with a rare class	rows, class proportions kept	it scatters groups to match proportions
grouped	new users, devices, patients, sites	whole groups	deployment is in the future too
temporal	the future	later periods	deployment is on new entities too (combine)

Key insight. Hold out the kind of novelty you will meet, measure what the costs care about, and let nothing from the held-out rows or from the future touch the pipeline.

How the professor asks it

Q. *A detector reports 99% accuracy on a dataset with 1% attacks. Explain what this number can and cannot mean, compute the balanced accuracy of the trivial classifier, and name the two metrics you would ask for.*

A. With $\pi = 0.01$, the classifier that always says "benign" has accuracy $1 - \pi = 0.99$ and recall 0: it detects nothing. So 99% accuracy is consistent with a detector that never fires. Its specificity is 1 and its recall 0, so its balanced accuracy is $(0 + 1)/2 = 0.5$, the number that exposes it. Ask for recall (fraction of attacks caught) and precision or false alarms per 100,000 benign flows (what each alarm costs an analyst); F_1 summarises the two but ignores true negatives.

Q. *Give an example of a feature that would yield near-perfect test performance and be useless in deployment, and explain the mechanism.*

A. "Number of analyst comments on the event": comments are written during the investigation that follows an intrusion, so the feature is a consequence of the label recorded after the fact. In the historical table it separates the classes almost perfectly; at decision time, before any investigation, its value is zero for every new event. This is leakage from the future; the rule for catching it is to ask, for each feature, at what moment its value becomes known relative to the moment of the decision.

7 Diagnosis: learning curves, and Lab 1

Learning curves

Vary the amount of training data and plot training and validation performance against it [on your sheet]. Patterns: both errors high and flat, underfitting (improve representation or capacity); training low and validation high, high variance (regularise, reduce capacity, more representative data); errors close and both still decreasing as N grows, data-limited (more representative examples will help); test performance degrading over calendar time, distribution shift (update data, time-aware evaluation). "Close and decreasing" means a small gap and both improving: not overfitting strongly, limited by evidence. Curves are diagnostic evidence, not proofs; read them with domain knowledge and error analysis. The full protocol and the sandwich argument for the two curves' shapes are in the Experiments document.

Lab 1 at a glance, mapped to the chapter

The intrusion-detection lab audits a labelled-flow dataset (imbalance, missing or non-finite values, duplicates, conflicting labels, redundancy); pre-declares random-row, grouped and chronological train/validation/test protocols (the split section); fits baselines (majority class, depth-3 tree, logistic regression, kNN) under each protocol; probes metadata shortcuts and shuffled labels (the shortcut probe); selects the decision threshold on validation (selection section); and reports test recall, false-positive rate and false alarms per 100,000 benign flows (the metrics). Every idea of this chapter appears there.

How the professor asks it

Q. *A learning curve shows both curves high, flat and close together from 2,000 to 40,000 rows. What do you conclude and what would you change?*
A. The procedure is not limited by data (more rows changed nothing) nor by variance (the gap is small): it is limited by bias, meaning the family or the representation cannot express the relationship. Add informative features, use a more flexible family, or reduce regularisation; collecting more of the same rows will not move the plateau.

Coverage of the textbook chapter

Textbook item (Chapter 3)	Where in this document
Definition: empirical risk, population risk, generalisation gap	§1 concept box
Proposition: training risk of a fitted model is optimistic	§1 math box, drill
Example: cows on grass, camels on sand; technique: weakening shortcuts	§1 shortcut probe, example
Definition: inductive bias	§1 concept box, versus box
Definition: capacity, nested families; proposition: training error favours capacity	§2
Definition: underfitting, overfitting; example: three polynomials; kNN direction; gotcha	§2
Definition: bias and variance; theorem and proof; example with numbers	§3
Remark: double descent; definition: three sources of excess error; classic questions	§3 concept box, drills
Definition: regularised ERM; theorem: soft constraint; ridge 1-D; lasso soft threshold; geometry figure	§4
Choosing λ; definition: early stopping; technique: regularisation in practice; classics	§4
Proposition: 2^d subsets; definition: filters and greedy searches; XOR example; leakage gotcha	§5
Three subsets; proposition: best of many validation scores; definition: K-fold CV	§5
Splits (random, stratified, grouped, temporal); intrusion example	§6
Proposition: accuracy hides minority failure; definition: metrics; table; proposition on F_1	§6
Definition: leakage; technique: preprocessing inside the pipeline; classics	§6
Definition: learning curves; diagnostic table; Lab 1	§7 and the Experiments document

The ten mistakes of this chapter

1. Reading a training score as evidence about new cases.
2. Not probing for shortcuts (new site, new period, shuffled labels, metadata removed).
3. Comparing families or choosing λ, k, depth, epochs by training error.
4. Talking about the bias or variance of one fitted model; expecting data to remove bias.
5. Getting the kNN direction backwards; treating zero training error as success.
6. Penalising unstandardised features or the intercept; linear grids for λ; last epoch instead of best.
7. Expecting ridge to select or lasso to be stable under correlation.
8. Selecting features on all rows before cross-validation; reporting the winner's validation score as final.
9. A random split for the future or for new entities; accuracy on imbalanced data.
10. Features known only after the outcome; preprocessing fitted on all rows; thresholds tuned on test.

Chapter 3 in ten lines

1. Gap $G = R(\hat{f}) - \widehat{R}_{\mathcal{D}}(\hat{f})$; $\mathbb{E}[\widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}})] \leq \min_{\mathcal{H}} R \leq \mathbb{E}[R(\hat{f}_{\mathcal{D}})]$: the sample chooses and grades.
2. Spurious cues fit the sample as well as stable ones; only new contexts separate them; inductive bias (data, family, regulariser, optimiser) decides off the sample.
3. Capacity: training error nonincreasing in nested families; validation U; underfitting both high, overfitting low/high; kNN capacity grows as k shrinks.
4. Squared error $= \sigma^2 + \text{Bias}^2 + \text{Var}$ by adding and subtracting f^* then $\bar{f}$; capacity trades them, data lowers variance only.
5. Regularised ERM = ERM over the nested family $\Omega \leq c$; training risk minimal at $\lambda = 0$, so λ is chosen on validation.
6. Ridge: factor $\sum x^2/(\sum x^2 + N\lambda)$, never zero. Lasso: $\text{sign}(z)\max(|z| - \lambda, 0)$, exact zeros, diamond corners,

unstable under correlation.

7. Early stopping: keep $\arg\min_t \widehat{R}_{\text{val}}(\hat{\theta}_t)$ with patience; training time is capacity.
8. 2^d subsets; filters miss interactions (XOR); greedy searches block; selection inside every fold or it leaks.
9. Best of many validation scores is optimistic; CV averages K honest fold scores; the test set is used once.
10. Split from the deployment question (random, stratified, grouped, temporal); metrics from the costs (recall, precision, specificity, balanced accuracy, F_1); no leakage; learning curves for bias/variance/data-limited.